

Kernel ERA: Model Specification, Estimation, Inference

Sunmee Kim

July 2026

1 Extended Redundancy Analysis

Let $\mathbf{Y}$ denote an $N \times Q$ outcome matrix, where N is the number of observations and Q is the number of outcome variables. The predictor variables are partitioned into K sets. Let $\mathbf{X}_k$ denote an $N \times P_k$ matrix containing the predictors in the k th predictor set, where $k = 1, \dots, K$, and $P = \sum_{k=1}^K P_k$. We assume that the columns of $\mathbf{Y}$ and $\mathbf{X}_k$ are standardized to have zero mean and unit variance.

Let x_{ikp} denote the value of the p th predictor within the k th predictor set for the i th observation. ERA defines the component score for the k th predictor set as

$$f_{ik} = \sum_{p=1}^{P_k} x_{ikp} w_{kp}, \quad (1)$$

where w_{kp} is the component weight assigned to x_{ikp} . In matrix form,

$$\mathbf{f}_k = \mathbf{X}_k \mathbf{w}_k, \quad (2)$$

where $\mathbf{f}_k$ is an $N \times 1$ vector of component scores and $\mathbf{w}_k$ is a $P_k \times 1$ vector of component weights. To remove the scale indeterminacy between $\mathbf{w}_k$ and the regression coefficients defined below, we impose the identification constraint

$$\text{Var}(\mathbf{f}_k) = 1, \quad k = 1, \dots, K,$$

which fixes the scale of each component and is enforced throughout estimation.

Let b_{kq} denote the regression coefficient relating the k th component to the q th outcome variable, and let

$$\mathbf{b}_k = [b_{k1} \ \cdots \ b_{kQ}]'.$$

The parameters in $\mathbf{w}_k$ and $\mathbf{b}_k$ are estimated by minimizing

$$f_{ERA} = SS \left(\mathbf{Y} - \sum_{k=1}^K \mathbf{X}_k \mathbf{w}_k \mathbf{b}_k' \right), \quad \text{subject to } \text{Var}(\mathbf{f}_k) = 1 \text{ for all } k, \quad (3)$$

where $SS(\mathbf{A}) = \text{tr}(\mathbf{A}'\mathbf{A})$. An alternating least-squares algorithm is used to minimize the objective function with respect to the unknown parameters in $\mathbf{w}_k$ and $\mathbf{b}_k$.

The goodness of fit of the model is measured by

$$FIT = 1 - \frac{SS \left(\mathbf{Y} - \sum_{k=1}^K \mathbf{X}_k \mathbf{w}_k \mathbf{b}_k' \right)}{SS(\mathbf{Y})}. \quad (4)$$

The FIT represents the proportion of the total variation in the outcome variables accounted for by the specified ERA model.

For a single outcome variable, the objective function reduces to

$$f_{ERA} = SS \left(y - \sum_{k=1}^K b_k \mathbf{X}_k \mathbf{w}_k \right). \quad (5)$$

With a single predictor set, it further reduces to

$$f_{ERA} = SS(y - b\mathbf{X}\mathbf{w}), \quad (6)$$

which is the unpenalized linear ERA objective corresponding to the Kernel ERA derivation.

2 Kernel Extended Redundancy Analysis

Kernel ERA preserves the prespecified predictor-set structure and the outcome model of linear ERA while allowing each predictor-set component to be a nonlinear function of its original predictors. Specifically, the construction of each component is extended from a linear combination of the original predictors to a linear combination of features in a potentially high-dimensional nonlinear feature space. The relationships between the resulting predictor-set components and the outcomes retain the standard ERA form.

2.1 The Model

Let

$$\mathbf{x}_{ik} = [x_{ik1} \quad \cdots \quad x_{ikP_k}]^\top \in \mathbb{R}^{P_k}$$

denote the predictor vector for the i th observation in the k th predictor set. Let

$$\phi_k : \mathbb{R}^{P_k} \longrightarrow \mathcal{H}_k$$

denote a nonlinear feature mapping from the original predictor space to a Hilbert feature space $\mathcal{H}_k$. The feature space may have a much larger dimension than the original predictor space and may even be infinite-dimensional.

Although a component is linear in the transformed features, it is generally a nonlinear function of the original predictors. For example, when $\mathbf{x} = (x_1, x_2)^\top$, the second-degree feature map

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$$

introduces squared and interaction features. Its feature-space inner product satisfies

$$\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle = (\mathbf{x}^\top \mathbf{z})^2.$$

Define the feature matrix as

$$\mathbf{\Phi}_k = \begin{bmatrix} \phi_k(\mathbf{x}_{1k})' \\ \vdots \\ \phi_k(\mathbf{x}_{Nk})' \end{bmatrix}.$$

Kernel ERA defines the component for the k th predictor set as

$$\mathbf{f}_k = \mathbf{\Phi}_k \mathbf{w}_k, \quad (7)$$

where $\mathbf{w}_k$ is a weight vector in $\mathcal{H}_k$.

Directly constructing $\mathbf{\Phi}_k$ and estimating $\mathbf{w}_k$ may be computationally impractical because $\mathcal{H}_k$ can be extremely high-dimensional or infinite-dimensional. Kernel ERA therefore seeks a representation of the component that depends only on inner products among the mapped observations rather than on the feature coordinates themselves.

Without loss of generality, $\mathbf{w}_k$ can be restricted to the linear span of the centered feature vectors. To see this, decompose $\mathbf{w}_k$ as

$$\mathbf{w}_k = \mathbf{w}_{k,\parallel} + \mathbf{w}_{k,\perp},$$

where

$$\mathbf{w}_{k,\parallel} \in \text{span} \{ \phi_k^c(\mathbf{x}_{1k}), \dots, \phi_k^c(\mathbf{x}_{Nk}) \}$$

and $\mathbf{w}_{k,\perp}$ is orthogonal to this span. Here $\phi_k^c(\mathbf{x}_{ik})$ denotes the centered feature vector. Since

$$\langle \phi_k^c(\mathbf{x}_{ik}), \mathbf{w}_{k,\perp} \rangle_{\mathcal{H}_k} = 0$$

for every observation, $\mathbf{w}_{k,\perp}$ makes no contribution to the component scores.

Consequently, the feature-space weight vector can be represented as

$$\mathbf{w}_k = \mathbf{\Phi}_k^\top \boldsymbol{\alpha}_k,$$

where $\boldsymbol{\alpha}_k$ is an $N \times 1$ vector of sample-space coefficients, or equivalently, coefficients in the kernel expansion. Substituting this into (7) gives

$$\mathbf{f}_k = \mathbf{\Phi}_k \mathbf{\Phi}_k^\top \boldsymbol{\alpha}_k = \mathbf{K}_k \boldsymbol{\alpha}_k, \quad (8)$$

where

$$\mathbf{K}_k = \mathbf{\Phi}_k \mathbf{\Phi}_k^\top$$

is the centered $N \times N$ Gram matrix for the k th predictor set.

Its (i, j) th element is

$$K_{k,ij} = \kappa_k(\mathbf{x}_{ik}, \mathbf{x}_{jk}) = \langle \phi_k(\mathbf{x}_{ik}), \phi_k(\mathbf{x}_{jk}) \rangle_{\mathcal{H}_k}. \quad (9)$$

If $\kappa_k(\cdot, \cdot)$ is a valid kernel function, $\mathbf{K}_k$ is positive semidefinite, and the feature mapping ϕ_k need not be evaluated explicitly. Common choices include the polynomial kernel,

$$\kappa_k(\mathbf{x}_{ik}, \mathbf{x}_{jk}) = (\mathbf{x}_{ik}' \mathbf{x}_{jk} + c)^d,$$

and the Gaussian kernel,

$$\kappa_k(\mathbf{x}_{ik}, \mathbf{x}_{jk}; \sigma) = \exp \left(-\frac{\|\mathbf{x}_{ik} - \mathbf{x}_{jk}\|_2^2}{2\sigma^2} \right), \quad \sigma > 0.$$

The kernel parameters, such as d , c , and σ , are specified or selected during model tuning.

The relationship between the predictor-set components and the outcomes remains the same as in linear ERA. Accordingly, **the kernel ERA model** is

$$\mathbf{Y} = \sum_{k=1}^K \mathbf{f}_k \mathbf{b}_k' + \mathbf{E} = \sum_{k=1}^K \mathbf{K}_k \boldsymbol{\alpha}_k \mathbf{b}_k' + \mathbf{E}. \quad (10)$$

2.2 Training-Sample Standardization and Kernel Centering

The implementation standardizes every predictor and outcome variable using statistics computed from the training sample. For a training variable $x_1, \dots, x_N$, define

$$\bar{x}_{\text{tr}} = \frac{1}{N} \sum_{i=1}^N x_i, \quad s_{x,\text{tr}} = \left\{ \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x}_{\text{tr}})^2 \right\}^{1/2}. \quad (11)$$

The standardized training values are

$$z_i = \frac{x_i - \bar{x}_{\text{tr}}}{s_{x,\text{tr}}}. \quad (12)$$

The divisor- N convention in (11) makes the empirical second moment of a centered standardized column equal to one. For a validation or test observation x_i^* , the transformation is

$$z_i^* = \frac{x_i^* - \bar{x}_{\text{tr}}}{s_{x,\text{tr}}}, \quad (13)$$

using the training mean and scale rather than statistics recomputed in the validation or test sample. The same rule is used for the outcome variables. Within cross-validation, “training sample” in (11)–(13) refers to the observations in the current analysis fold.

Let $\tilde{\mathbf{K}}_k$ denote the raw training Gram matrix for predictor set k , and let

$$\mathbf{H}_N = \mathbf{I}_N - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^\top. \quad (14)$$

The Gram matrix used in estimation is the feature-centered matrix

$$\mathbf{K}_k = \mathbf{H}_N \tilde{\mathbf{K}}_k \mathbf{H}_N. \quad (15)$$

Thus, all occurrences of $\mathbf{K}_k$ below refer to the centered Gram matrix. Centering in (15) is equivalent to subtracting the training feature-space mean from every mapped observation.

For M new observations, let $\tilde{\mathbf{K}}_{k,*}$ be the $M \times N$ raw cross-kernel matrix between the new and training observations. Define

$$\mathbf{r}_{k,*} = \frac{1}{N} \tilde{\mathbf{K}}_{k,*} \mathbf{1}_N, \quad \mathbf{c}_k = \frac{1}{N} \tilde{\mathbf{K}}_k^\top \mathbf{1}_N, \quad \bar{\kappa}_k = \frac{1}{N^2} \mathbf{1}_N^\top \tilde{\mathbf{K}}_k \mathbf{1}_N. \quad (16)$$

The centered cross-kernel matrix is

$$\mathbf{K}_{k,*} = \tilde{\mathbf{K}}_{k,*} - \mathbf{r}_{k,*} \mathbf{1}_N^\top - \mathbf{1}_M \mathbf{c}_k^\top + \bar{\kappa}_k \mathbf{1}_M \mathbf{1}_N^\top. \quad (17)$$

Equation (17) ensures that predictions for new observations are expressed relative to the feature-space origin defined by the training sample.

2.3 Parameter Estimation

2.3.1 Penalized Objective Function

Let

$$\mathbf{F} = [\mathbf{f}_1 \quad \dots \quad \mathbf{f}_K], \quad \mathbf{f}_k = \mathbf{K}_k \boldsymbol{\alpha}_k,$$

and define

$$\mathbf{B} = \begin{bmatrix} \mathbf{b}_1^\top \\ \vdots \\ \mathbf{b}_K^\top \end{bmatrix},$$

where $\mathbf{b}_k$ is the $Q \times 1$ vector of coefficients relating the k th predictor-set component to the Q outcome variables. The Kernel ERA model can then be expressed as

$$\mathbf{Y} = \mathbf{F}\mathbf{B} + \mathbf{E} = \sum_{k=1}^K \mathbf{K}_k \boldsymbol{\alpha}_k \mathbf{b}_k^\top + \mathbf{E}. \quad (18)$$

For fixed kernel parameters, the unknown parameters $\boldsymbol{\alpha}_k$, $k = 1, \dots, K$, and $\mathbf{B}$ are estimated by minimizing the penalized least-squares criterion

$$\begin{aligned} \mathcal{L}_{\text{KERA}}(\{\boldsymbol{\alpha}_k\}_{k=1}^K, \mathbf{B}) = & \left\| \mathbf{Y} - \sum_{k=1}^K \mathbf{K}_k \boldsymbol{\alpha}_k \mathbf{b}_k^\top \right\|_F^2 \\ & + \lambda \sum_{k=1}^K \boldsymbol{\alpha}_k^\top \mathbf{K}_k \boldsymbol{\alpha}_k, \end{aligned} \quad (19)$$

subject to

$$\frac{1}{N} \|\mathbf{K}_k \boldsymbol{\alpha}_k\|_2^2 = 1, \quad k = 1, \dots, K. \quad (20)$$

The penalty in (19) corresponds to the squared norm of the feature-space weight vector. Specifically, because

$$\mathbf{w}_k = \boldsymbol{\Phi}_k^\top \boldsymbol{\alpha}_k,$$

it follows that

$$\begin{aligned} \|\mathbf{w}_k\|_{\mathcal{H}_k}^2 &= \boldsymbol{\alpha}_k^\top \boldsymbol{\Phi}_k \boldsymbol{\Phi}_k^\top \boldsymbol{\alpha}_k \\ &= \boldsymbol{\alpha}_k^\top \mathbf{K}_k \boldsymbol{\alpha}_k. \end{aligned} \quad (21)$$

Thus, the ridge penalty controls the complexity of the component function in the feature space.

2.3.2 Alternating Least-Squares Algorithm

An alternating least-squares algorithm is used to estimate $\{\boldsymbol{\alpha}_k\}_{k=1}^K$ and $\mathbf{B}$. For given values of the kernel parameters and ridge parameter λ , the algorithm alternates between the following two parameter-update steps until convergence.

Initialization. Initial component scores are obtained from a linear ERA solution. Let

$$\mathbf{F}^{(0)} = \begin{bmatrix} \mathbf{f}_1^{(0)} & \dots & \mathbf{f}_K^{(0)} \end{bmatrix}$$

denote the resulting initial component-score matrix. Each component is normalized such that

$$\left\| \mathbf{f}_k^{(0)} \right\|_2^2 = N.$$

Given $\mathbf{F}^{(0)}$, the initial outcome coefficient matrix is computed as

$$\mathbf{B}^{(0)} = \left[\left(\mathbf{F}^{(0)} \right)^\top \mathbf{F}^{(0)} \right]^{-1} \left(\mathbf{F}^{(0)} \right)^\top \mathbf{Y}. \quad (22)$$

This linear warm start supplies both of the quantities that Step 1 needs to begin: $\mathbf{B}^{(0)}$ furnishes $\mathbf{b}_k$ (and hence $\mathbf{b}_k^\top \mathbf{b}_k$) for the first update of $\boldsymbol{\alpha}_k$ in (28), and $\mathbf{F}^{(0)}$ enters the first partial-residual matrix $\mathbf{R}_k$ in (23). Consequently, the dual coefficients $\boldsymbol{\alpha}_k$ themselves never need to be initialized directly – a convenient feature, since they live in the typically much higher-dimensional (and, for kernels such as the Gaussian kernel, infinite-dimensional) dual representation of $\mathcal{H}_k$.

Step 1: Update the kernel coefficients α_k for fixed $\mathbf{B}$. Conditional on the current value of $\mathbf{B}$ and the component scores from the other predictor sets, the kernel coefficients are updated one predictor set at a time.

For the k th predictor set, define the partial residual matrix

$$\mathbf{R}_k = \mathbf{Y} - \sum_{\ell \neq k} \mathbf{f}_\ell \mathbf{b}_\ell^\top. \quad (23)$$

The part of the objective function that depends on α_k is

$$\mathcal{L}_k(\alpha_k) = \left\| \mathbf{R}_k - \mathbf{K}_k \alpha_k \mathbf{b}_k^\top \right\|_F^2 + \lambda \alpha_k^\top \mathbf{K}_k \alpha_k. \quad (24)$$

Expanding the squared-error term gives

$$\begin{aligned} \mathcal{L}_k(\alpha_k) = & \text{tr}(\mathbf{R}_k^\top \mathbf{R}_k) - 2 \alpha_k^\top \mathbf{K}_k \mathbf{R}_k \mathbf{b}_k \\ & + (\mathbf{b}_k^\top \mathbf{b}_k) \alpha_k^\top \mathbf{K}_k^2 \alpha_k + \lambda \alpha_k^\top \mathbf{K}_k \alpha_k. \end{aligned} \quad (25)$$

Differentiating (25) with respect to α_k yields

$$\begin{aligned} \frac{1}{2} \frac{\partial \mathcal{L}_k}{\partial \alpha_k} = & - \mathbf{K}_k \mathbf{R}_k \mathbf{b}_k \\ & + (\mathbf{b}_k^\top \mathbf{b}_k) \mathbf{K}_k^2 \alpha_k + \lambda \mathbf{K}_k \alpha_k. \end{aligned} \quad (26)$$

Setting the derivative equal to zero gives

$$\mathbf{K}_k \left[\left\{ (\mathbf{b}_k^\top \mathbf{b}_k) \mathbf{K}_k + \lambda \mathbf{I}_N \right\} \alpha_k - \mathbf{R}_k \mathbf{b}_k \right] = \mathbf{0}. \quad (27)$$

The provisional dual-coefficient update used by the algorithm is therefore

$$\tilde{\alpha}_k = \left[(\mathbf{b}_k^\top \mathbf{b}_k) \mathbf{K}_k + \lambda \mathbf{I}_N \right]^{-1} \mathbf{R}_k \mathbf{b}_k. \quad (28)$$

The provisional coefficient vector is then rescaled to impose the component normalization constraint:

$$\alpha_k = \frac{\sqrt{N} \tilde{\alpha}_k}{\|\mathbf{K}_k \tilde{\alpha}_k\|_2}. \quad (29)$$

The updated component score is

$$\mathbf{f}_k = \mathbf{K}_k \alpha_k. \quad (30)$$

By construction,

$$\mathbf{f}_k^\top \mathbf{f}_k = N,$$

and hence

$$\frac{1}{N} \mathbf{f}_k^\top \mathbf{f}_k = 1.$$

The updates in Step 1 are performed sequentially for $k = 1, \dots, K$. Consequently, when updating α_k , the partial residual in (23) uses the most recently updated component scores for predictor sets $1, \dots, k-1$ and the component scores from the preceding iteration for predictor sets $k+1, \dots, K$. Thus, Step 1 follows a Gauss–Seidel block-update scheme.

When there is a single outcome variable ($Q = 1$), $\mathbf{b}_k$ reduces to the scalar b_k , so that $\mathbf{b}_k^\top \mathbf{b}_k = b_k^2$, and $\mathbf{R}_k \mathbf{b}_k$ reduces to $b_k \mathbf{R}_k$, with $\mathbf{R}_k$ itself an $N \times 1$ vector in this case. The provisional update in (28) then simplifies to

$$\tilde{\alpha}_k = (b_k^2 \mathbf{K}_k + \lambda \mathbf{I}_N)^{-1} b_k \mathbf{R}_k, \quad (31)$$

which is the update implemented in the ALS code and coincides, for $K = 1$, with the closed-form solution $\hat{\alpha} = (b^2 \mathbf{K} + \lambda \mathbf{I})^{-1} b \mathbf{R}$ obtained directly from the single-component Lagrangian.

Step 2: Update the outcome coefficients $\mathbf{B}$ for fixed $\{\alpha_k\}_{k=1}^K$. After all K component scores have been updated, let

$$\mathbf{F} = [\mathbf{f}_1 \quad \cdots \quad \mathbf{f}_K].$$

For fixed $\mathbf{F}$, the penalty term in (19) does not depend on $\mathbf{B}$. Therefore, updating $\mathbf{B}$ is equivalent to minimizing

$$\mathcal{L}_{\mathbf{B}}(\mathbf{B}) = \|\mathbf{Y} - \mathbf{FB}\|_F^2. \quad (32)$$

Differentiating (32) with respect to $\mathbf{B}$ gives

$$\frac{\partial \mathcal{L}_{\mathbf{B}}}{\partial \mathbf{B}} = -2\mathbf{F}^\top \mathbf{Y} + 2\mathbf{F}^\top \mathbf{FB}. \quad (33)$$

Setting the derivative equal to zero yields the normal equations

$$\mathbf{F}^\top \mathbf{FB} = \mathbf{F}^\top \mathbf{Y}. \quad (34)$$

Provided that $\mathbf{F}^\top \mathbf{F}$ is nonsingular, the updated outcome coefficient matrix is

$$\hat{\mathbf{B}} = \left(\mathbf{F}^\top \mathbf{F}\right)^{-1} \mathbf{F}^\top \mathbf{Y}. \quad (35)$$

When $Q = 1$, $\mathbf{B}$ reduces to the $K \times 1$ vector of regression coefficients used in the ALS code, and $\mathbf{F}^\top \mathbf{F}$ is simply the $K \times K$ Gram matrix of the component scores.

If $\mathbf{F}^\top \mathbf{F}$ is singular, the corresponding Moore–Penrose generalized-inverse solution may be used.

Convergence. Let $\mathbf{B}^{(t)}$ denote the outcome-coefficient matrix after iteration t , and let $\hat{\mathcal{L}}_{\text{KERA}}^{(t)}$ denote the penalized criterion in (19) evaluated after all component and coefficient updates in that iteration. The implementation monitors the total absolute change in the outcome coefficients,

$$\Delta_B^{(t)} = \sum_{k=1}^K \sum_{q=1}^Q \left| b_{kq}^{(t)} - b_{kq}^{(t-1)} \right|, \quad (36)$$

and the relative objective change

$$\Delta_{\mathcal{L}}^{(t)} = \frac{\left| \hat{\mathcal{L}}_{\text{KERA}}^{(t-1)} - \hat{\mathcal{L}}_{\text{KERA}}^{(t)} \right|}{\max \left\{ 1, \left| \hat{\mathcal{L}}_{\text{KERA}}^{(t-1)} \right| \right\}}. \quad (37)$$

A run is declared converged when either

$$\Delta_B^{(t)} \leq \epsilon_B, \quad (38)$$

or

$$\Delta_B^{(t)} \leq 10\epsilon_B \quad \text{and} \quad \Delta_{\mathcal{L}}^{(t)} \leq \epsilon_{\mathcal{L}} \quad (39)$$

is satisfied. If neither condition is met before the maximum number of iterations, the returned solution is retained for diagnostic purposes but is flagged as nonconverged. In the current simulation implementation, $\epsilon_B = 10^{-6}$, $\epsilon_{\mathcal{L}} = 10^{-9}$, and the maximum number of iterations is 500.

At the final iterate, the penalized criterion is

$$\hat{\mathcal{L}}_{\text{KERA}} = \left\| \mathbf{Y} - \hat{\mathbf{F}}\hat{\mathbf{B}} \right\|_F^2 + \lambda \sum_{k=1}^K \hat{\boldsymbol{\alpha}}_k^\top \mathbf{K}_k \hat{\boldsymbol{\alpha}}_k. \quad (40)$$

2.3.3 Spectral Computation of the Kernel-Coefficient Update

The implementation solves the update in (28) in the positive-eigenvalue subspace of the centered Gram matrix. Let

$$\mathbf{K}_k = \mathbf{U}_k \mathbf{D}_k \mathbf{U}_k^\top, \quad (41)$$

where $\mathbf{D}_k = \text{diag}(d_{k1}, \dots, d_{kr_k})$ contains the numerically positive eigenvalues retained after a rank tolerance is applied. With

$$q_k = \mathbf{b}_k^\top \mathbf{b}_k, \quad \mathbf{u}_k = \mathbf{R}_k \mathbf{b}_k,$$

the RKHS-penalized provisional update is computed as

$$\tilde{\boldsymbol{\alpha}}_k = \mathbf{U}_k \text{diag} \left(\frac{1}{q_k d_{k1} + \lambda}, \dots, \frac{1}{q_k d_{kr_k} + \lambda} \right) \mathbf{U}_k^\top \mathbf{u}_k. \quad (42)$$

This range-space solution avoids numerical inversion in the null space of $\mathbf{K}_k$ and yields the minimum-norm kernel representation relevant to the component scores. A direct positive-semidefinite solve is available as a numerical alternative. If the provisional component is numerically degenerate, the first nonzero kernel principal component is used as a fallback and is normalized to have squared norm N .

2.3.4 Multiple Initializations and Selection of the Final Solution

Because the complete Kernel ERA criterion is not jointly convex in the kernel coefficients and outcome coefficients, the final fit is evaluated from multiple initializations. The first run uses the linear ERA warm start described above. Each additional run begins from a linear ERA solution with randomly initialized component weights. Let $\hat{\mathcal{L}}_s$ and C_s denote the final objective and convergence indicator for start s , $s = 1, \dots, S$. The selected start is

$$\hat{s} = \underset{s: C_s=1}{\text{argmin}} \hat{\mathcal{L}}_s. \quad (43)$$

Thus, only converged solutions are compared whenever at least one start converges. If every start fails to converge, the finite solution with the smallest objective is retained solely as an operational output and the final fit remains flagged as nonconverged. The current final-model implementation uses $S = 3$: one linear warm start and two random starts. Cross-validation fits use one linear warm start per candidate to control computation time.

2.3.5 Tuning-Parameter Selection by Convergence-Restricted V -Fold Cross-Validation

The primary implementation selects the ridge parameter λ and the Gaussian bandwidth σ by V -fold cross-validation rather than by the projection-based generalized cross-validation criterion. A common (λ, σ) pair is shared across all K predictor sets. Let Λ and $\mathcal{S}$ denote finite grids for λ and σ , respectively, and let $\mathcal{I}_v$ denote validation fold v , with $N_v = |\mathcal{I}_v|$.

For each $(\lambda, \sigma) \in \Lambda \times \mathcal{S}$ and each fold v , the predictor and outcome standardization constants, raw Gram matrices, and kernel-centering quantities are estimated using only the analysis-fold observations $\mathcal{I}_{-v}$. The fitted model is then applied to $\mathcal{I}_v$ using the analysis-fold standardization and centering quantities. For a Q -variate outcome, the validation error is

$$\text{MSE}_v(\lambda, \sigma) = \frac{1}{N_v Q} \left\| \mathbf{Y}_v - \hat{\mathbf{Y}}_v^{(-v)}(\lambda, \sigma) \right\|_F^2. \quad (44)$$

The cross-validated error and its fold-based standard error are

$$\text{CV}(\lambda, \sigma) = \frac{1}{V} \sum_{v=1}^V \text{MSE}_v(\lambda, \sigma), \quad (45)$$

$$\text{SE}_{\text{CV}}(\lambda, \sigma) = \frac{\text{SD} \{\text{MSE}_1(\lambda, \sigma), \dots, \text{MSE}_V(\lambda, \sigma)\}}{\sqrt{V}}. \quad (46)$$

A candidate is eligible for selection only if it produces a finite validation error and the ALS algorithm converges in every fold. Define

$$A(\lambda, \sigma) = \prod_{v=1}^V \mathbb{I} \{\text{MSE}_v(\lambda, \sigma) < \infty \text{ and } C_v(\lambda, \sigma) = 1\}, \quad (47)$$

where $C_v(\lambda, \sigma)$ is the fold-specific convergence indicator. The selected tuning parameters are

$$(\hat{\lambda}, \hat{\sigma}) = \underset{(\lambda, \sigma) \in \Lambda \times \mathcal{S}: A(\lambda, \sigma) = 1}{\text{argmin}} \text{CV}(\lambda, \sigma). \quad (48)$$

The unrestricted minimum over candidates with complete finite fold errors is also retained as a diagnostic. If that unrestricted minimum fails the convergence requirement, it is reported but excluded from selection. If no candidate is eligible, the condition-replication fit terminates with a tuning error rather than silently selecting a nonconverged candidate.

After tuning, Kernel ERA is refitted to the complete training sample using $(\hat{\lambda}, \hat{\sigma})$ and the multiple-start procedure in (43). In the current implementation, $V = 5$ and the minimum eligible cross-validated error is the primary selection rule. The one-standard-error rule is available as a sensitivity analysis but is not used for the primary simulation results.

2.4 Prediction for New Observations

Let $\mathbf{X}_{k,*}$ denote a new predictor matrix for set k . Its columns are standardized using the training-sample means and scales in (13). The centered cross-kernel matrix $\mathbf{K}_{k,*}$ is then formed using (17). The predicted component scores are

$$\hat{\mathbf{f}}_{k,*} = \mathbf{K}_{k,*} \hat{\boldsymbol{\alpha}}_k, \quad (49)$$

and, with $\hat{\mathbf{F}}_* = [\hat{\mathbf{f}}_{1,*} \ \dots \ \hat{\mathbf{f}}_{K,*}]$, the predicted standardized outcomes are

$$\hat{\mathbf{Y}}_{*,\text{std}} = \hat{\mathbf{F}}_* \hat{\mathbf{B}}. \quad (50)$$

Predictions on the original outcome scale are obtained by reversing the training-sample outcome standardization. No centering or scaling constants are estimated from the validation or test observations.

2.5 Model Assessment and Operational Stability

For the standardized training outcome, the training FIT is

$$\text{FIT}_{\text{train}} = 1 - \frac{\|\mathbf{Y}_{\text{std}} - \hat{\mathbf{F}}\hat{\mathbf{B}}\|_F^2}{\|\mathbf{Y}_{\text{std}}\|_F^2}. \quad (51)$$

For the single-outcome simulation, out-of-sample performance is assessed by

$$\text{MSE}_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} (y_i - \hat{y}_i)^2, \quad (52)$$

$$R_{\text{test}}^2 = 1 - \frac{\sum_{i \in \text{test}} (y_i - \hat{y}_i)^2}{\sum_{i \in \text{test}} (y_i - \bar{y}_{\text{train}})^2}, \quad (53)$$

with test RMSE, mean absolute error, and the correlation between y_i and $\hat{y}_i$ also recorded. The denominator in (53) uses the training-sample outcome mean so that the reference predictor is defined without using test-sample information. The train-test FIT gap is

$$G_{\text{fit}} = \text{FIT}_{\text{train}} - R_{\text{test}}^2. \quad (54)$$

Algorithmic stability is treated as an estimand of the simulation rather than as a preprocessing filter. A final run is classified as interpolating when

$$\text{FIT}_{\text{train}} > 0.98. \quad (55)$$

Define

$$S_{\text{stable}} = \mathbb{I} \{ \text{final fit converged and did not satisfy (55)} \}, \quad (56)$$

$$S_{\text{failure}} = \mathbb{I} \{ \text{final fit did not converge or satisfied (55)} \}. \quad (57)$$

Condition-specific nonconvergence, interpolation, and operational-failure rates are reported directly. Predictive and recovery summaries are produced both for all operational outputs and conditionally for replications with $S_{\text{stable}} = 1$. The conditional analysis retains the corresponding Linear ERA and oracle results from the same Monte Carlo replications, thereby preserving paired comparisons. Nonconverged Kernel ERA solutions are not automatically replaced by a more strongly regularized tuning candidate, because such replacement would alter the prespecified estimation procedure and conceal finite-sample instability.

3 Monte Carlo Simulation Study

3.1 Predictor Generation

The main simulation contains $K = 3$ prespecified predictor sets. Let g_i be a global latent variable shared by all predictors, let h_{ik} be a set-specific latent variable, and let e_{ikp} be predictor-specific noise, all mutually independent standard normal variables. Predictor p in set k is generated as

$$x_{ikp} = \sqrt{\rho_{\text{between}}} g_i + \sqrt{\rho_{\text{within}} - \rho_{\text{between}}} h_{ik} + \sqrt{1 - \rho_{\text{within}}} e_{ikp}. \quad (58)$$

Consequently, two distinct predictors in the same set have correlation ρ_{within} , whereas predictors in different sets have correlation ρ_{between} . The main design uses $\rho_{\text{within}} = 0.4$ and $\rho_{\text{between}} = 0.1$.

3.2 True Predictor-Set Components and Outcome Generation

For a generic standardized predictor block $\mathbf{x} = (x_1, x_2, x_3, \dots)^\top$, define the following component functions:

$$g_{\text{L}}(\mathbf{x}) = 0.80x_1 - 0.60x_2 + 0.40x_3, \quad (59)$$

$$g_{\text{C}}(\mathbf{x}) = 0.80(x_1^2 - 1) + 0.55(x_2^2 - 1) + 0.85x_1x_2 + 0.35 \sin(1.3x_3), \quad (60)$$

$$g_{\text{T}}(\mathbf{x}) = 0.90 \tanh(1.2x_1) + 0.75 \{ \mathbb{I}(x_2 > 0) - 0.5 \} + 0.55x_1x_3, \quad (61)$$

$$g_{\text{R}}(\mathbf{x}) = 1.10 \exp \{ -0.65(x_1^2 + x_2^2) \} + 0.45 \sin(x_3). \quad (62)$$

The three data-generating scenarios are

$$\text{linear:} \quad (g_1, g_2, g_3) = (g_L, g_L, g_L), \quad (63)$$

$$\text{mixed:} \quad (g_1, g_2, g_3) = (g_L, g_C, g_T), \quad (64)$$

$$\text{nonlinear:} \quad (g_1, g_2, g_3) = (g_R, g_C, g_T). \quad (65)$$

The raw true components are standardized using only the training sample and the divisor- N rule in (11). Let $\mathbf{F}_0$ denote the resulting true-component matrix. With

$$\boldsymbol{\beta} = [0.80 \quad 0.65 \quad 0.50]^\top, \quad (66)$$

the signal is

$$\eta_i = \mathbf{f}_{0i}^\top \boldsymbol{\beta}. \quad (67)$$

Let

$$\sigma_{\eta, \text{tr}}^2 = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} (\eta_i - \bar{\eta}_{\text{tr}})^2. \quad (68)$$

To attain a target population signal ratio R_0^2 , the error variance is set to

$$\sigma_\varepsilon^2 = \sigma_{\eta, \text{tr}}^2 \frac{1 - R_0^2}{R_0^2}, \quad (69)$$

and the outcome is generated as

$$y_i = \eta_i + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\varepsilon^2). \quad (70)$$

3.3 Factorial Design and Tuning Grids

The main simulation crosses three functional-form scenarios, two training sample sizes, two predictor-set dimensions, and two target signal levels:

$$3 \times 2 \times 2 \times 2 = 24 \text{ conditions}, \quad (71)$$

where

$$N_{\text{train}} \in \{150, 300\}, \quad P_k \in \{5, 20\}, \quad R_0^2 \in \{0.30, 0.60\}.$$

Each condition uses $N_{\text{test}} = 1000$ independently generated test observations. The final design uses 200 Monte Carlo replications per condition; smaller pilot runs use the same conditions and seeds with fewer replications.

The primary ridge grid is

$$\Lambda = \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}. \quad (72)$$

For $P_k = 5$, the base Gaussian-bandwidth grid is

$$\mathcal{C} = \{0.75, 1.5, 3, 6, 12, 24, 48, 96\}. \quad (73)$$

To retain a comparable distance scale when the predictor-set dimension changes, the candidate bandwidths are

$$\mathcal{S}(P_k) = \left\{ c \sqrt{\frac{P_k}{5}} : c \in \mathcal{C} \right\}. \quad (74)$$

Thus, the $P_k = 20$ grid equals $\{1.5, 3, 6, 12, 24, 48, 96, 192\}$. This scaling adjusts the search grid for the approximate $\sqrt{P_k}$ growth of Euclidean distances but does not remove small-bandwidth candidates. Instability observed in difficult small-sample, larger-dimension conditions is therefore retained as a simulation outcome.

3.4 Component Recovery and Coefficient Evaluation

Because a component and its outcome coefficient may be multiplied jointly by -1 without changing the fitted contribution, estimated component signs are aligned using the training-sample true components. For predictor set k , define

$$s_k = \text{sign} \left\{ \text{cor} \left(\hat{\mathbf{f}}_{k,\text{tr}}, \mathbf{f}_{0k,\text{tr}} \right) \right\}, \quad (75)$$

with $s_k = 1$ when the correlation is zero or undefined. The estimated training and test components and the corresponding outcome coefficient are multiplied by s_k .

Component recovery is summarized by the test-sample correlation between the aligned estimated and true components. Because correlation is invariant to location and scale, a separately calibrated RMSE is also recorded. Let (a_k, c_k) minimize

$$\sum_{i \in \text{train}} \left[f_{0ik} - \left(a_k + c_k s_k \hat{f}_{ik} \right) \right]^2. \quad (76)$$

The calibrated test RMSE is

$$\text{RMSE}_{f_k} = \left\{ \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} \left[f_{0ik} - \left(\hat{a}_k + \hat{c}_k s_k \hat{f}_{ik} \right) \right]^2 \right\}^{1/2}. \quad (77)$$

Outcome-coefficient estimates are compared with $\beta_k/s_{y,\text{tr}}$, the true coefficient on the standardized outcome scale, after sign alignment.

3.5 Reporting Strategy

For each condition, the simulation reports tuning eligibility, the rate at which the unrestricted CV minimum is excluded for nonconvergence, final-fit convergence, interpolation, operational failure, selected-grid boundaries, and computation time. Prediction and component-recovery metrics are summarized in two complementary ways:

1. *All operational outputs*: every completed replication is retained, including a finite output from a nonconverged final run.
2. *Stable-replication analysis*: results are restricted to Monte Carlo replications for which the Kernel ERA final fit converged and was not classified as interpolating. Linear ERA and oracle results are restricted to the same replication keys so that method comparisons remain paired.

The first analysis characterizes the performance of the complete operational procedure, whereas the second describes predictive and recovery performance conditional on successful estimation. The distinction prevents algorithmic failure from being hidden while also preventing a small number of unstable fits from obscuring the behavior of successfully estimated models.